

CineVault Rentals

MySQL Database Analyst Assessment

SQL Assignment Submission

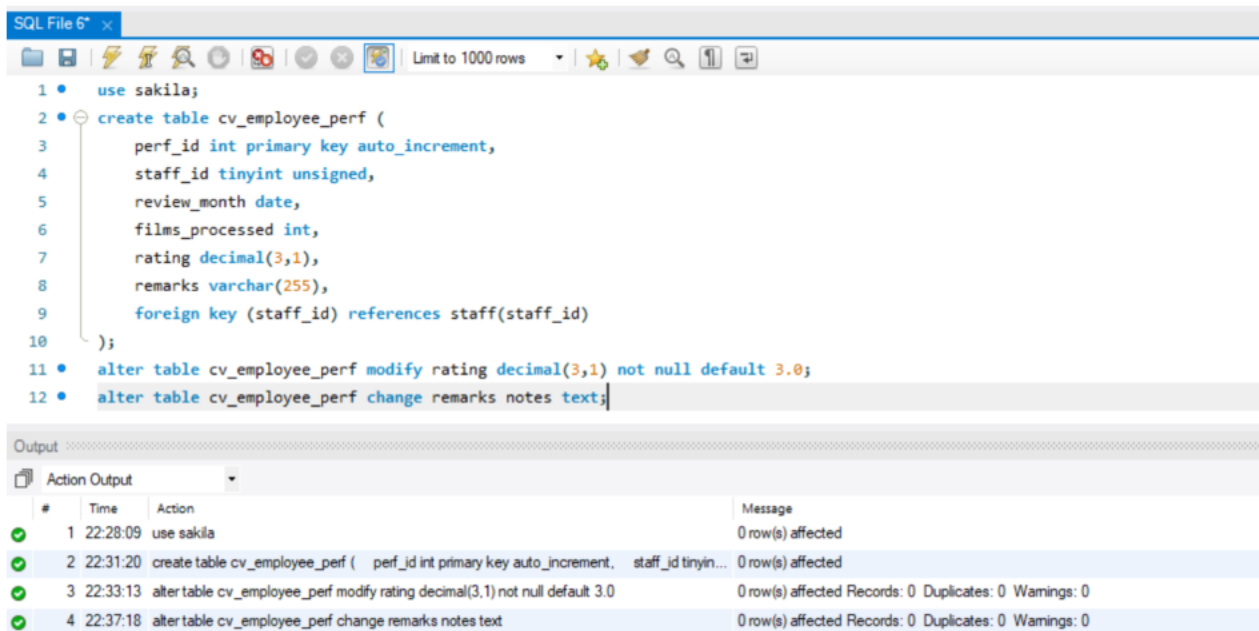
Prepared by: Nagarjuna

Database: sakila

Q1. New Franchise Rollout

Part A

I created the cv_employee_perf table with the required performance fields. The table uses perf_id as the primary key with auto increment, while staff_id stores the related staff record.



```
1 • use sakila;
2 • create table cv_employee_perf (
3     perf_id int primary key auto_increment,
4     staff_id tinyint unsigned,
5     review_month date,
6     films_processed int,
7     rating decimal(3,1),
8     remarks varchar(255),
9     foreign key (staff_id) references staff(staff_id)
10 );
11 • alter table cv_employee_perf modify rating decimal(3,1) not null default 3.0;
12 • alter table cv_employee_perf change remarks notes text;
```

Output

#	Time	Action	Message
✓ 1	22:28:09	use sakila	0 row(s) affected
✓ 2	22:31:20	create table cv_employee_perf (perf_id int primary key auto_increment, staff_id tinyin...	0 row(s) affected
✓ 3	22:33:13	alter table cv_employee_perf modify rating decimal(3,1) not null default 3.0	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✓ 4	22:37:18	alter table cv_employee_perf change remarks notes text	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Part B

I added a foreign key on staff_id that references staff(staff_id). This keeps the performance records connected to valid staff records.

Part C

I changed rating so it cannot contain NULL values and set 3.0 as its default value.

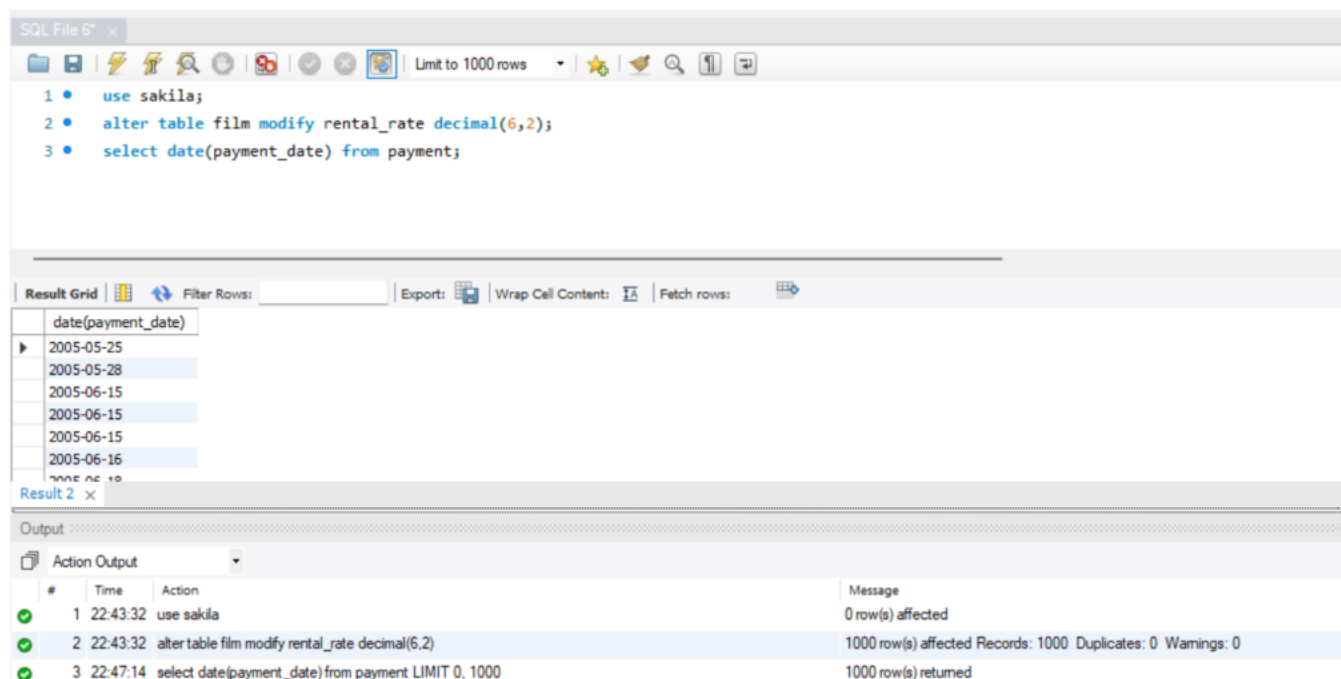
Part D

I renamed remarks to notes and changed its data type to TEXT. Using the cv prefix keeps the original Sakila tables protected.

Q2. Data Type Conversion Issue

Part A

I widened film.rental_rate from DECIMAL(4,2) to DECIMAL(6,2). This allows values up to 999.99 while preserving the existing data.



The screenshot shows a SQL client interface with a script named 'SQL File 6'. The script contains three commands:

- 1 • `use sakila;`
- 2 • `alter table film modify rental_rate decimal(6,2);`
- 3 • `select date(payment_date) from payment;`

Below the script, the 'Result Grid' shows the output of the third command, displaying a list of dates from 2005-05-25 to 2005-06-16. The 'Output' pane shows the execution log for these commands:

#	Time	Action	Message
1	22:43:32	use sakila	0 row(s) affected
2	22:43:32	alter table film modify rental_rate decimal(6,2)	1000 row(s) affected Records: 1000 Duplicates: 0 Warnings: 0
3	22:47:14	select date(payment_date) from payment LIMIT 0, 1000	1000 row(s) returned

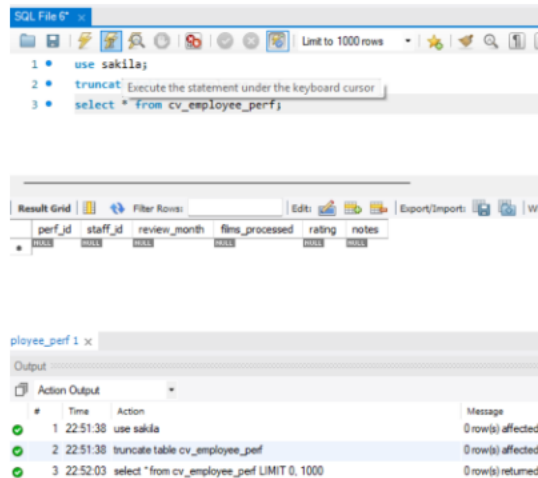
Part B

I converted payment_date from DATETIME to a DATE value for display only. The stored DATETIME values in the payment table were not changed.

Q3. Decommissioning and Cleanup

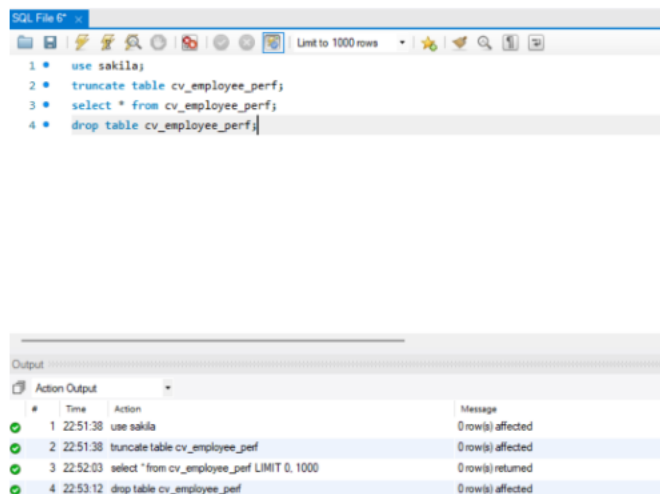
Part A

I used TRUNCATE to remove all rows from cv_employee_perf while keeping the table structure available.



Part B

I used DROP TABLE after the pilot was permanently cancelled. This removes the table and its structure completely.



Part C

For Part A, TRUNCATE is the better choice because the goal is to remove all rows while keeping the table available for future use.

It is designed for clearing the entire table quickly while preserving its structure for later inserts or reuse.

Although DELETE can also remove every row, it is more useful when specific records must be removed using a WHERE condition.

For Part B, DROP TABLE is the better choice because the pilot was permanently cancelled and the table is no longer required.

DROP removes the table definition as well as its data, so no unused table is left behind.

Using DELETE here would only empty the table and would leave an unnecessary structure that has already served its purpose.

Q4. Normalization Check

Part A

The proposed rental_log_flat design does not satisfy the intended 1NF requirement in this assignment because it mixes repeating customer and film details with rental data. This repetition creates redundant data and increases the chance of inconsistent values when the same information is updated. It can also cause update, insert, and delete anomalies. A normalized design stores each type of information in the appropriate table and links the tables with keys.

Part B

Sakila avoids these problems by separating customer, film, rental, and inventory information into related tables.

The customer table stores customer details using customer_id, while the film table stores film details using film_id.

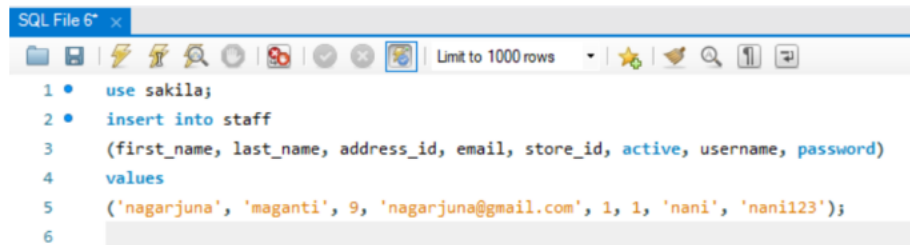
The rental and inventory tables connect customers to physical film copies through their keys, reducing repeated information.

This structure improves consistency and makes inserts, updates, and deletes easier to manage because each fact is maintained in one appropriate location.

Q5. Onboarding New Staff

Part A

I inserted a new staff member by explicitly listing the columns being supplied. The selected address_id and store_id values refer to existing records, so the foreign key relationships remain valid. The Workbench result confirms that the staff record was inserted successfully.



```
SQL File 6
1 • use sakila;
2 • insert into staff
3 (first_name, last_name, address_id, email, store_id, active, username, password)
4 values
5 ('nagarjuna', 'maganti', 9, 'nagarjuna@gmail.com', 1, 1, 'nani', 'nani123');
6
```

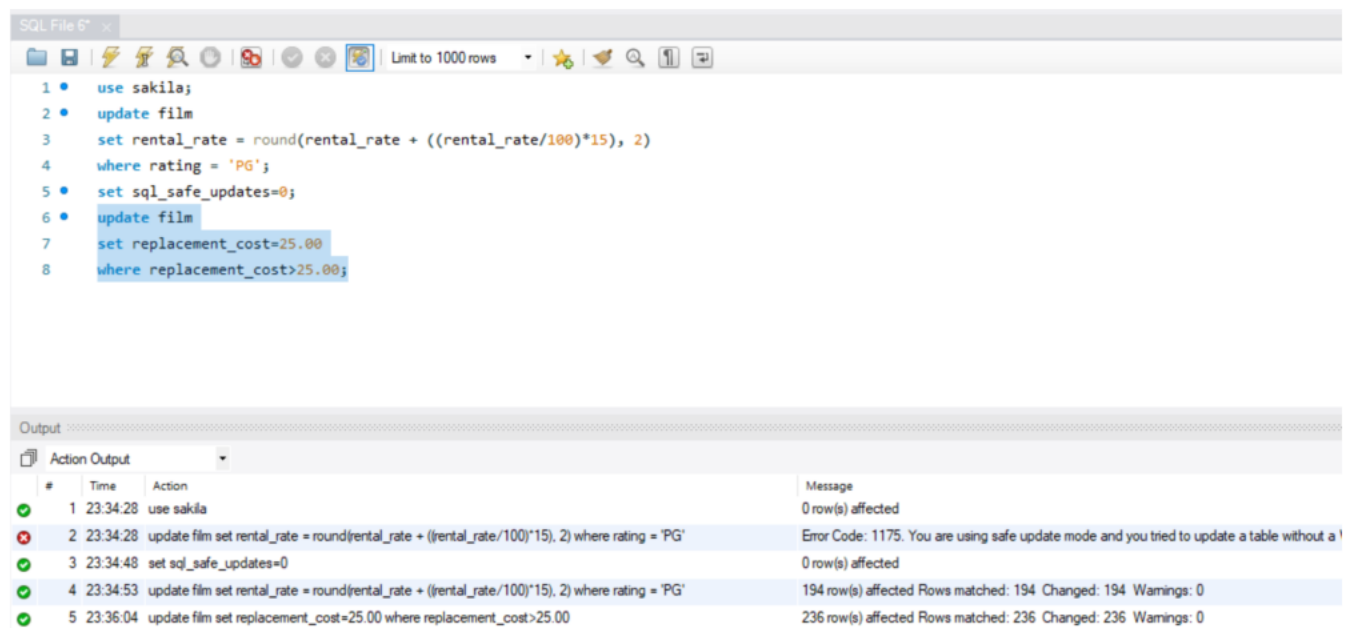


Output			
Action Output			
#	Time	Action	Message
✓ 1	23:06:35	use sakila	0 row(s) affected
✓ 2	23:06:35	insert into staff (first_name, last_name, address_id, email, store_id, active, u...	1 row(s) affected

Q6. Price Correction

Part A

I increased the rental rate of every PG rated film by 15 percent and rounded the result to two decimal places.



The screenshot displays a SQL editor window titled "SQL File 6*" with the following SQL script:

```
1 • use sakila;
2 • update film
3   set rental_rate = round(rental_rate + ((rental_rate/100)*15), 2)
4   where rating = 'PG';
5 • set sql_safe_updates=0;
6 • update film
7   set replacement_cost=25.00
8   where replacement_cost>25.00;
```

Below the editor, the "Output" pane shows the "Action Output" table:

#	Time	Action	Message
✓ 1	23:34:28	use sakila	0 row(s) affected
✗ 2	23:34:28	update film set rental_rate = round(rental_rate + ((rental_rate/100)*15), 2) where rating = 'PG'	Error Code: 1175. You are using safe update mode and you tried to update a table without a 1
✓ 3	23:34:48	set sql_safe_updates=0	0 row(s) affected
✓ 4	23:34:53	update film set rental_rate = round(rental_rate + ((rental_rate/100)*15), 2) where rating = 'PG'	194 row(s) affected Rows matched: 194 Changed: 194 Warnings: 0
✓ 5	23:36:04	update film set replacement_cost=25.00 where replacement_cost>25.00	236 row(s) affected Rows matched: 236 Changed: 236 Warnings: 0

Part B

I capped replacement_cost at 25.00 only for films whose existing replacement cost was above 25.00. The Workbench output confirms both update operations.

Q7. Data Cleanup

Part A

The cv_employee_perf table had already been dropped in Q3. I recreated the table and inserted test records so the required conditional delete could be demonstrated.

```
SQL File 0
1 use sakila;
2 create table cv_employee_perf (
3     perf_id int primary key auto_increment,
4     staff_id tinyint unsigned,
5     review_month date,
6     films_processed int,
7     rating decimal(3,1) not null default 3.0,
8     notes text,
9     foreign key (staff_id) references staff(staff_id)
10 );
11 insert into cv_employee_perf
12 (staff_id, review_month, films_processed, rating, notes)
13 values
14 (1, '2026-09-01', 0, 1.5, 'test data'),
15 (2, '2026-09-01', 0, 1.0, 'junk data'),
16 (1, '2026-09-01', 10, 1.5, 'valid data'),
17 (2, '2026-09-01', 0, 3.0, 'valid data');
```

Output

#	Time	Action	Message
1	23:40:54	use sakila;	0 row(s) affected
2	23:40:54	create table cv_employee_perf (perf_id int primary key auto_increment, staff_id tinyint unsigned, rev...	0 row(s) affected
3	23:42:10	insert into cv_employee_perf (staff_id, review_month, films_processed, rating, notes) values (1, 2026-09-01, ...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

Part A continued

I deleted the records where films_processed was 0 and rating was below 2.0. The valid records remained after the cleanup.

```
SQL File 0
1 use sakila;
2 delete from cv_employee_perf
3 where films_processed = 0
4 and rating < 2.0;
5 set sql_safe_updates=0;
6 select * from cv_employee_perf;
```

Result Grid

perf_id	staff_id	review_month	films_processed	rating	notes
3	1	2026-09-01	10	1.5	valid data
4	2	2026-09-01	0	3.0	valid data

Output

#	Time	Action	Message
1	23:43:52	use sakila;	0 row(s) affected
2	23:43:52	delete from cv_employee_perf where films_processed = 0 and rating < 2.0	Error Code: 1175. You are usin
3	23:44:09	set sql_safe_updates=0;	0 row(s) affected
4	23:44:16	delete from cv_employee_perf where films_processed = 0 and rating < 2.0	2 row(s) affected
5	23:44:36	select * from cv_employee_perf LIMIT 0, 1000	2 row(s) returned

Part B

Before running DELETE without a WHERE clause in production, I would first verify the table, the intended rows, and the exact statement being executed.

A missing WHERE condition can remove every row from the table, so the command should be reviewed carefully before it is run.

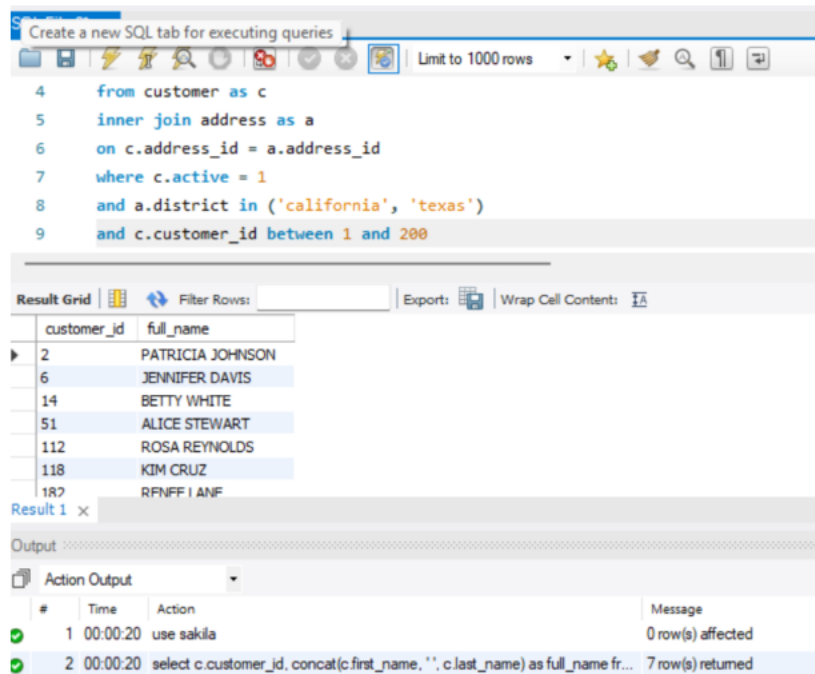
I would also take a backup or confirm that a reliable recovery point is available so the data can be restored if the deletion is accidental.

For a high-risk cleanup, I would test the statement in a safe environment first and confirm the affected row count before executing it in production.

Q8. Marketing Segment 1

Part A

I joined customer with address because the district is stored in the address table. I selected active customers, limited the district to California or Texas, and restricted customer_id to the range from 1 to 200.



The screenshot shows a SQL query editor interface. At the top, there's a toolbar with icons for saving, undo, redo, and other functions. Below the toolbar, the SQL query is displayed in a monospaced font. The query is as follows:

```
4  from customer as c
5  inner join address as a
6  on c.address_id = a.address_id
7  where c.active = 1
8  and a.district in ('california', 'texas')
9  and c.customer_id between 1 and 200
```

Below the query, there's a section labeled "Result Grid". It shows a table with two columns: "customer_id" and "full_name". The table contains 7 rows of data:

customer_id	full_name
2	PATRICIA JOHNSON
6	JENNIFER DAVIS
14	BETTY WHITE
51	ALICE STEWART
112	ROSA REYNOLDS
118	KIM CRUZ
182	RFNFF I ANF

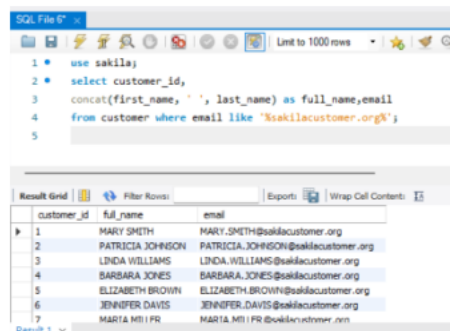
Below the result grid, there's an "Output" section. It shows a table with three columns: "#", "Time", and "Action". The table contains two rows of data:

#	Time	Action	Message
1	00:00:20	use sakila	0 row(s) affected
2	00:00:20	select c.customer_id, concat(c.first_name, ' ', c.last_name) as full_name fr...	7 row(s) returned

Q9. Marketing Segment 2

Part A

I used LIKE to find customer email addresses containing sakilacustomer.org.

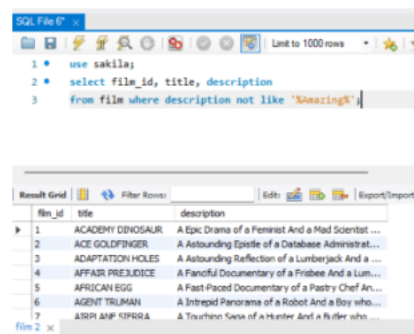


```
1 use sakila;
2 select customer_id,
3 concat(first_name, ' ', last_name) as full_name, email
4 from customer where email like '%sakilacustomer.org%';
5
```

customer_id	full_name	email
1	MARY SMITH	MARY.SMITH@sakilacustomer.org
2	PATRICIA JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org
3	LINDA WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org
4	BARBARA JONES	BARBARA.JONES@sakilacustomer.org
5	ELIZABETH BROWN	ELIZABETH.BROWN@sakilacustomer.org
6	JENNIFER DAVIS	JENNIFER.DAVIS@sakilacustomer.org
7	MARTA MIFFER	MARTA.MIFFER@sakilacustomer.org

Part B

I selected films whose descriptions do not contain the word Amazing.

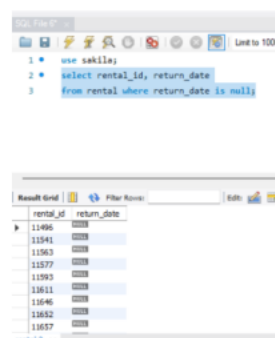


```
1 use sakila;
2 select film_id, title, description
3 from film where description not like '%Amazing%';
```

film_id	title	description
1	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...
2	ACE GOLDFINGER	A Astounding Espionage of a Database Administrat...
3	ADAPTATION HOLES	A Astounding Reflection of a Lumberjack And a Lum...
4	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...
5	AFRICAN EGG	A Fast-Paced Documentary of a Pastry Chef An...
6	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...
7	AIROPOLIS	A Touching Saga of a Hunter And a Ruler who...

Part C

I used IS NULL on return_date to identify rental records that do not have a recorded return date.



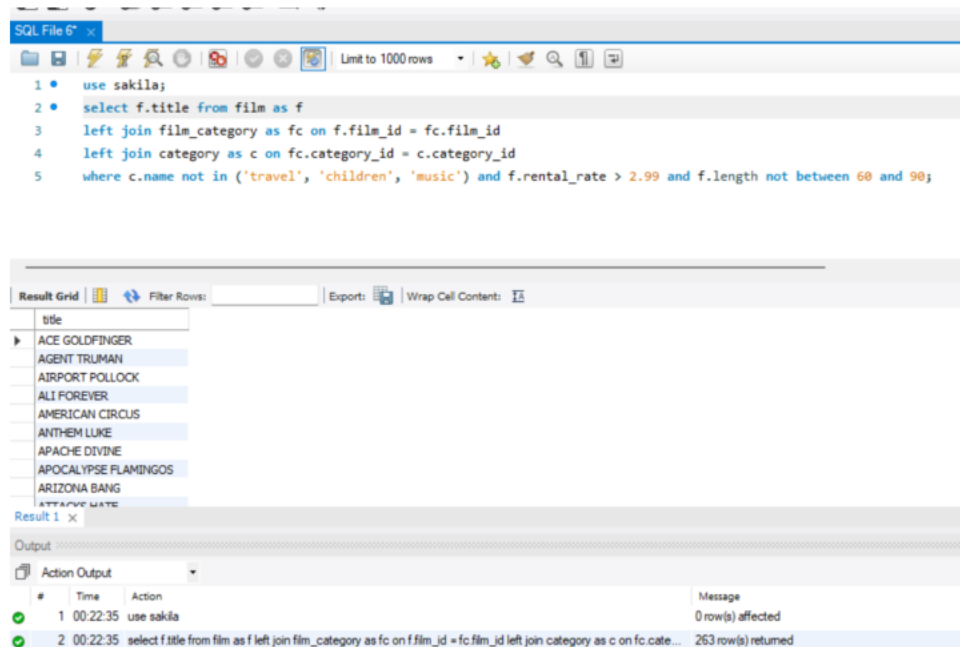
```
1 use sakila;
2 select rental_id, return_date
3 from rental where return_date is null;
```

rental_id	return_date
11486	
11541	
11563	
11577	
11593	
11611	
11646	
11652	
11657	

Q10. Operator Combination Challenge

Part A

I joined film, film_category, and category so the film titles could be filtered by category. I excluded Children, Music, and Travel, required rental_rate to be greater than 2.99, and excluded films with a length between 60 and 90 minutes. The Workbench returned 263 matching rows.



The screenshot displays the MySQL Workbench interface. At the top, the SQL editor shows a query with five lines: using the sakila database, selecting film titles, left joining film_category, left joining category, and filtering by category name (excluding travel, children, music), rental rate, and film length.

```
1 • use sakila;
2 • select f.title from film as f
3   left join film_category as fc on f.film_id = fc.film_id
4   left join category as c on fc.category_id = c.category_id
5   where c.name not in ('travel', 'children', 'music') and f.rental_rate > 2.99 and f.length not between 60 and 90;
```

Below the editor, the 'Result Grid' tab is active, showing a list of film titles. The 'Output' tab is also visible, showing the execution log with two actions: using the sakila database and executing the main query, which returned 263 rows.

#	Time	Action	Message
1	00:22:35	use sakila	0 row(s) affected
2	00:22:35	select f.title from film as f left join film_category as fc on f.film_id = fc.film_id left join category as c on fc.cate...	263 row(s) returned

Q11. Inner Join Core Report

Part A

I used inner joins between film, film_category, category, and language. The query returns the film title, category name, and language name for films that have all required relationships.

The screenshot shows a SQL IDE interface with three tabs: 'SQL File 6*', 'SQL File 6*', and 'SQL File 7*'. The active tab contains the following SQL query:

```
1 • use sakila;
2 • select f.title,c.name as category,l.name as language from film as f
3 inner join film_category as fc on f.film_id = fc.film_id
4 inner join category as c on fc.category_id = c.category_id
5 inner join language as l on f.language_id=l.language_id;
```

Below the query editor, the 'Result Grid' is displayed, showing a table with three columns: 'title', 'category', and 'language'. The table contains 10 rows of data:

title	category	language
AMADEUS HOLY	Action	English
AMERICAN CIRCUS	Action	English
ANTITRUST TOMATOES	Action	English
ARK RIDGEMONT	Action	English
BAREFOOT MANCHURIAN	Action	English
BERETS AGENT	Action	English
BRIDE INTRIGUE	Action	English
BULL SHAWSHANK	Action	English
CADDYSHACK JEDI	Action	English
PRINCE OF REMEMBRANCE	Action	English

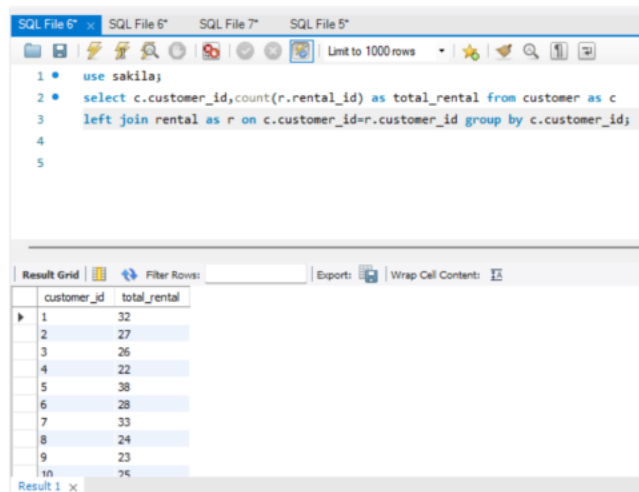
Below the result grid, the 'Output' section is visible, showing the execution log:

#	Time	Action	Message
1	00:28:44	use sakila	0 row(s) affected
2	00:28:44	select f.title,c.name as category,l.name as language from film as f inner join film_category as fc on f.film_id = ...	1000 row(s) returned

Q12. Left and Right Join Gap Analysis

Part A

For the left join, I started from customer so every customer remains in the result, including customers with no rentals. Counting rental_id gives a count of zero when there is no matching rental.



The screenshot shows a SQL IDE window with four tabs: SQL File 6*, SQL File 6*, SQL File 7*, and SQL File 5*. The active tab is SQL File 6*, which contains the following SQL code:

```
1 • use sakila;
2 • select c.customer_id, count(r.rental_id) as total_rental from customer as c
3   left join rental as r on c.customer_id=r.customer_id group by c.customer_id;
4
5
```

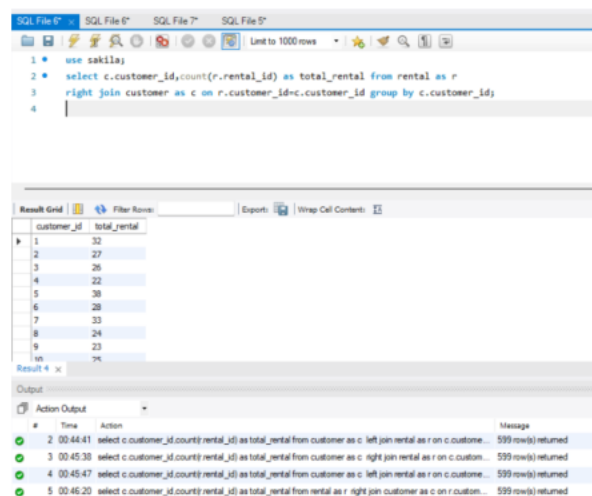
Below the code editor is the 'Result Grid' tab, which displays the results of the query. The grid has two columns: 'customer_id' and 'total_rental'. The results are as follows:

customer_id	total_rental
1	32
2	27
3	26
4	22
5	38
6	28
7	33
8	24
9	23
10	25

The bottom of the window shows 'Result 1 x'.

Part B

I rewrote the same logic using a right join, starting from rental and preserving the customer table. Both versions returned 599 customer rows.



The screenshot shows a SQL IDE window with four tabs: SQL File 6*, SQL File 6*, SQL File 7*, and SQL File 5*. The active tab is SQL File 6*, which contains the following SQL code:

```
1 • use sakila;
2 • select c.customer_id, count(r.rental_id) as total_rental from rental as r
3   right join customer as c on r.customer_id=c.customer_id group by c.customer_id;
4
```

Below the code editor is the 'Result Grid' tab, which displays the results of the query. The grid has two columns: 'customer_id' and 'total_rental'. The results are as follows:

customer_id	total_rental
1	32
2	27
3	26
4	22
5	38
6	28
7	33
8	24
9	23
10	25

The bottom of the window shows 'Result 4 x'.

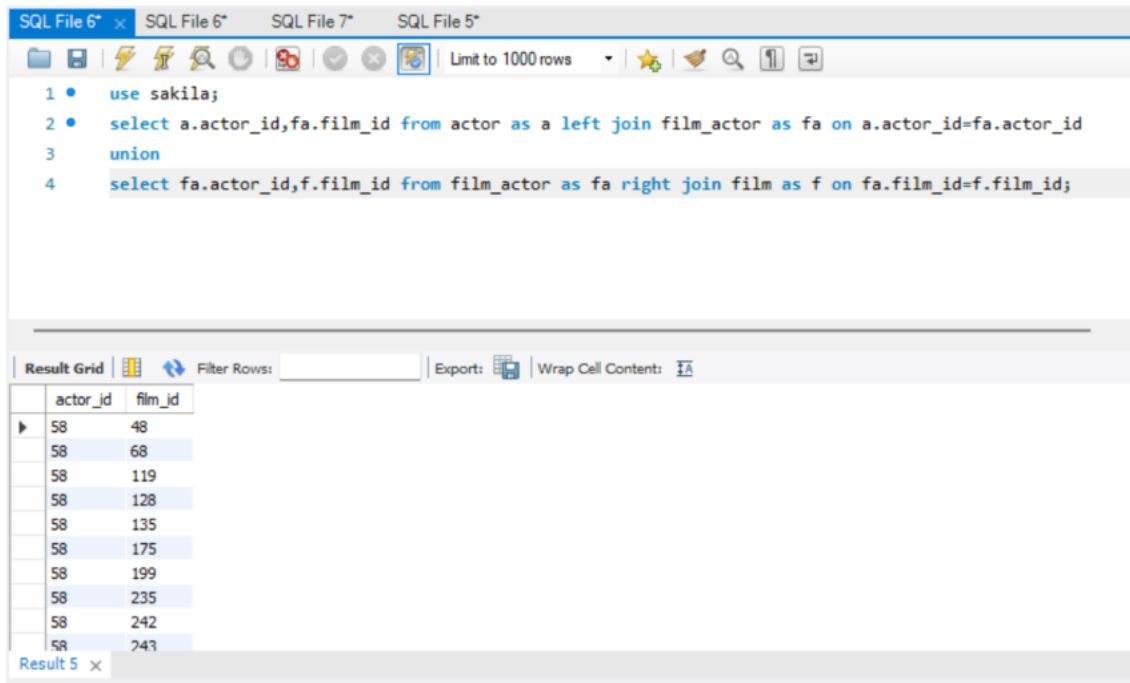
Below the result grid is the 'Output' tab, which displays the execution log. The log shows the following actions and messages:

#	Time	Action	Message
2	00:44:41	select c.customer_id, count(r.rental_id) as total_rental from customer as c left join rental as r on c.customer_id=r.customer_id group by c.customer_id;	599 row(s) returned
3	00:45:38	select c.customer_id, count(r.rental_id) as total_rental from customer as c right join rental as r on c.customer_id=r.customer_id group by c.customer_id;	599 row(s) returned
4	00:45:47	select c.customer_id, count(r.rental_id) as total_rental from customer as c left join rental as r on c.customer_id=r.customer_id group by c.customer_id;	599 row(s) returned
5	00:46:20	select c.customer_id, count(r.rental_id) as total_rental from rental as r right join customer as c on r.customer_id=c.customer_id group by c.customer_id;	599 row(s) returned

Q13. Full Outer Join Simulation

Part A

MySQL does not provide a native full outer join, so I simulated it using a left join, UNION, and right join. One side keeps actors even when they have no linked film, while the other keeps films even when they have no linked actor. UNION combines the two sides and removes duplicate rows.



The screenshot shows a SQL IDE with four tabs: 'SQL File 6*', 'SQL File 6*', 'SQL File 7*', and 'SQL File 5*'. The active tab 'SQL File 6*' contains the following SQL query:

```
1 • use sakila;
2 • select a.actor_id,fa.film_id from actor as a left join film_actor as fa on a.actor_id=fa.actor_id
3 union
4 select fa.actor_id,f.film_id from film_actor as fa right join film as f on fa.film_id=f.film_id;
```

Below the query editor, the 'Result Grid' tab is active, displaying the results of the query. The results are shown in a table with two columns: 'actor_id' and 'film_id'. The table contains 10 rows of data, where the first 8 rows have 'actor_id' 58 and various 'film_id' values, and the last 2 rows have 'actor_id' 58 and 'film_id' 242.

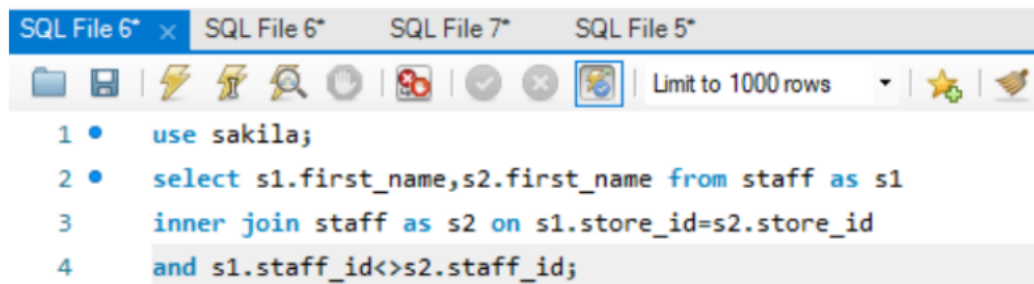
actor_id	film_id
58	48
58	68
58	119
58	128
58	135
58	175
58	199
58	235
58	242
58	243

At the bottom left of the result grid, there is a tab labeled 'Result 5' with a close button (X).

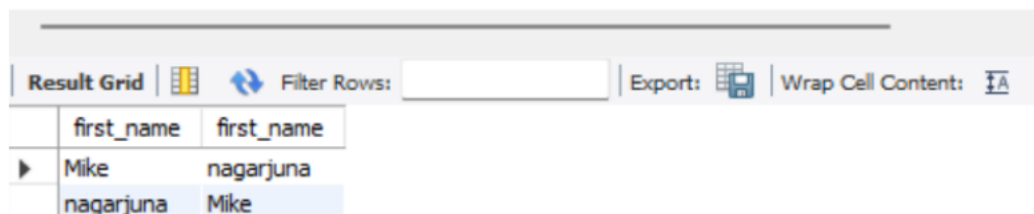
Q14. Self Join Peer Comparison

Part A

I joined the staff table to itself using two aliases. Matching store_id identifies staff members who work at the same store, while comparing staff_id prevents a staff member from being paired with themselves.



```
1 • use sakila;  
2 • select s1.first_name,s2.first_name from staff as s1  
3 inner join staff as s2 on s1.store_id=s2.store_id  
4 and s1.staff_id<>s2.staff_id;
```



	first_name	first_name
▶	Mike	nagarjuna
	nagarjuna	Mike

Q15. Cross Join Scenario Planning

Part A

I used a cross join between store and category to create every possible store and film category combination. This produces the promotion matrix requested by Marketing. A cross join should be used cautiously on large tables because every row from one table is combined with every row from the other table, which can make the result very large.

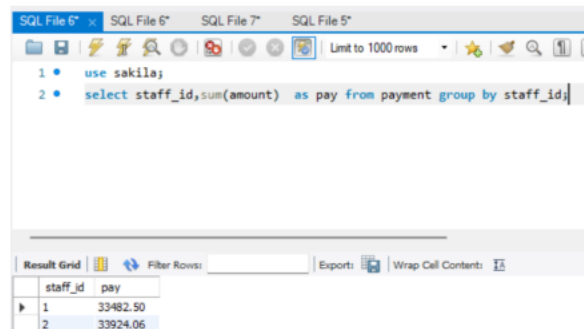
```
SQL File 6* x SQL File 6* SQL File 7* SQL File 5*
Limit to 1000 rows
1 • use sakila;
2 • select s.store_id,c.name from store as s cross join category as c;
```

store_id	name
2	Action
1	Action
2	Animation
1	Animation
2	Children
1	Children
2	Classics
1	Classics
2	Comedy

Q16. Revenue Report

Part A

I grouped payment records by staff_id and calculated the total payment collected by each staff member.



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

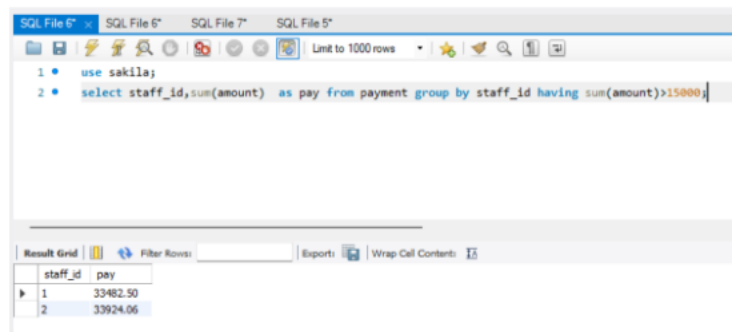
```
1 • use sakila;
2 • select staff_id, sum(amount) as pay from payment group by staff_id;
```

The result grid displays the following data:

staff_id	pay
1	33462.50
2	33924.06

Part B

I used HAVING to keep only staff members whose total revenue exceeded 15000.



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

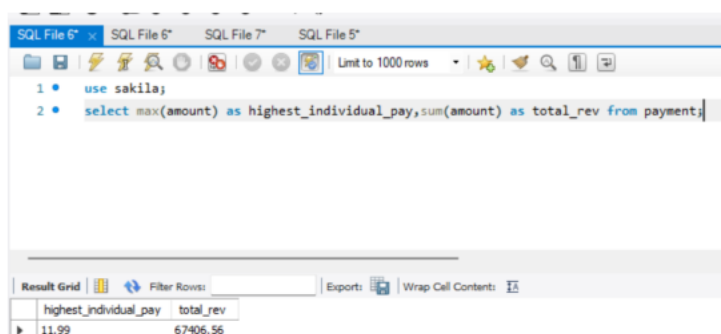
```
1 • use sakila;
2 • select staff_id, sum(amount) as pay from payment group by staff_id having sum(amount) > 15000;
```

The result grid displays the following data:

staff_id	pay
1	33462.50
2	33924.06

Part C

I calculated the overall company revenue and the highest individual payment without GROUP BY, producing one overall summary row.



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
1 • use sakila;
2 • select max(amount) as highest_individual_pay, sum(amount) as total_rev from payment;
```

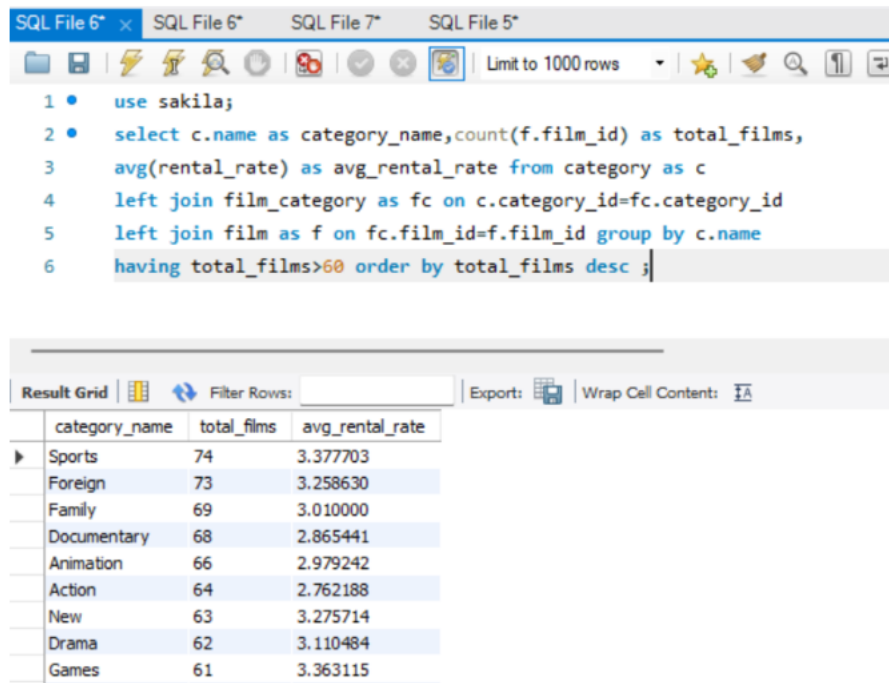
The result grid displays the following data:

highest_individual_pay	total_rev
11.99	67406.56

Q17. Output Based Question

Part A

I reverse engineered the requested result by grouping films by category, counting the films, and calculating the average rental rate. I then kept categories with more than 60 films and sorted them by total film count in descending order. The exact values can vary with the Sakila data snapshot, but the query structure follows the requirement.



The screenshot shows a SQL IDE with four tabs: SQL File 6*, SQL File 6*, SQL File 7*, and SQL File 5*. The active tab is SQL File 6*. The query editor contains the following SQL code:

```
1 • use sakila;
2 • select c.name as category_name, count(f.film_id) as total_films,
3     avg(rental_rate) as avg_rental_rate from category as c
4     left join film_category as fc on c.category_id=fc.category_id
5     left join film as f on fc.film_id=f.film_id group by c.name
6     having total_films>60 order by total_films desc ;
```

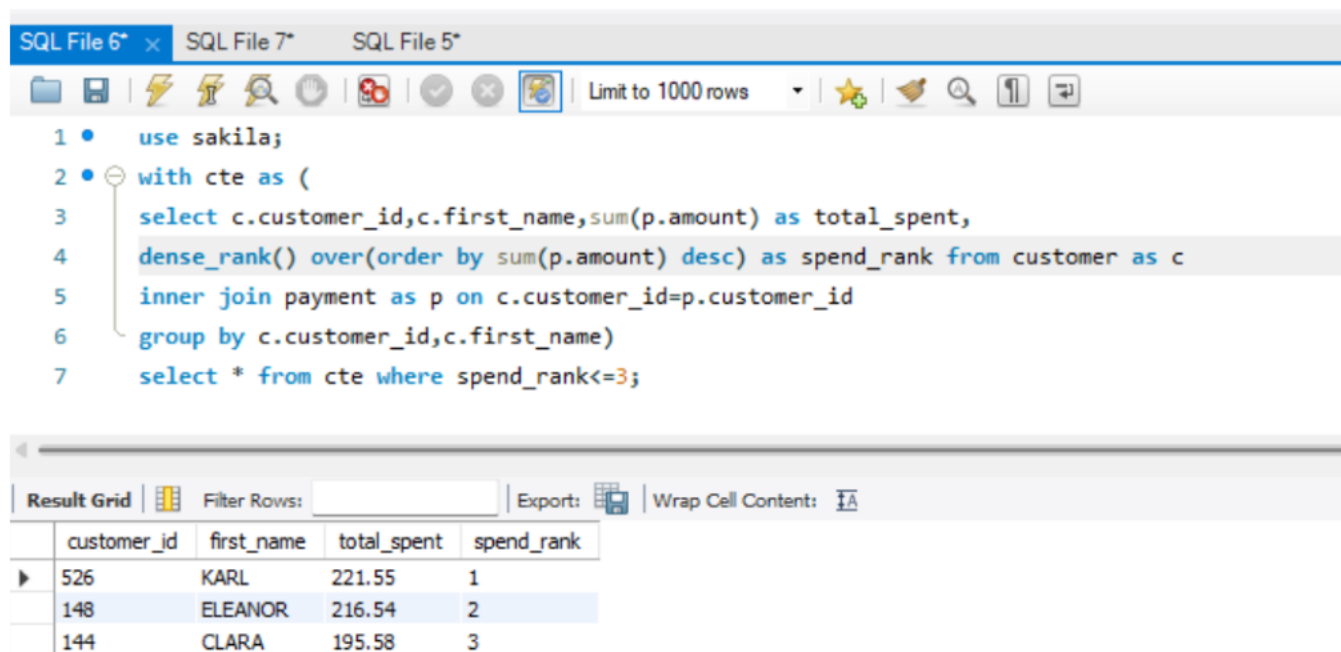
Below the query editor is the Result Grid. It has a toolbar with icons for grid, table, and chart views, a Filter Rows input, an Export button, and a Wrap Cell Content checkbox. The grid displays the following data:

	category_name	total_films	avg_rental_rate
▶	Sports	74	3.377703
	Foreign	73	3.258630
	Family	69	3.010000
	Documentary	68	2.865441
	Animation	66	2.979242
	Action	64	2.762188
	New	63	3.275714
	Drama	62	3.110484
	Games	61	3.363115

Q18. Rank the Customers

Part A

I calculated each customer's total payment and used DENSE_RANK to rank customers from the highest spender downward. DENSE_RANK gives tied customers the same rank and does not leave gaps after a tie, which matches the requirement. A CTE was used so the ranked result could be filtered to the required top ranks.



The screenshot shows a SQL IDE interface with three tabs: 'SQL File 6*', 'SQL File 7*', and 'SQL File 5*'. The active tab is 'SQL File 6*'. The query editor displays the following SQL code:

```
1 • use sakila;
2 • with cte as (
3   select c.customer_id,c.first_name,sum(p.amount) as total_spent,
4   dense_rank() over(order by sum(p.amount) desc) as spend_rank from customer as c
5   inner join payment as p on c.customer_id=p.customer_id
6   group by c.customer_id,c.first_name)
7   select * from cte where spend_rank<=3;
```

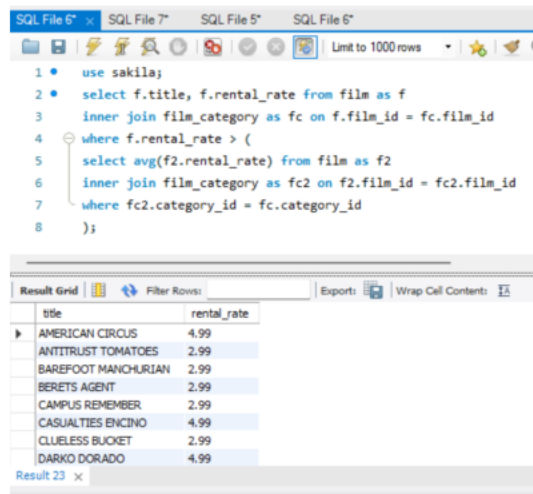
Below the query editor, the 'Result Grid' is visible. It shows the results of the query, with columns: customer_id, first_name, total_spent, and spend_rank. The results are as follows:

customer_id	first_name	total_spent	spend_rank
526	KARL	221.55	1
148	ELEANOR	216.54	2
144	CLARA	195.58	3

Q19. Subqueries

Part A

I used a correlated subquery to compare each film's rental rate with the average rental rate of its own category. The inner query references the category of the current film.



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

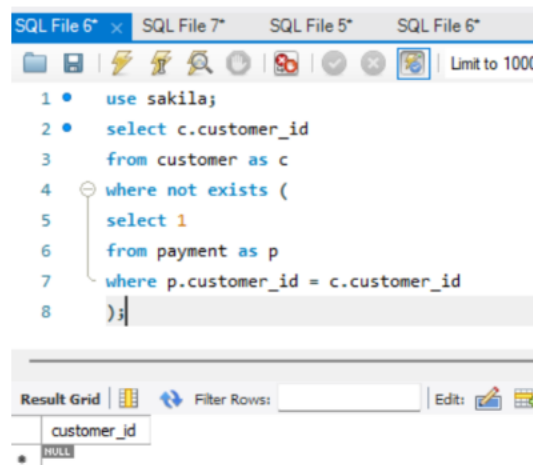
```
1 • use sakila;
2 • select f.title, f.rental_rate from film as f
3   inner join film_category as fc on f.film_id = fc.film_id
4   where f.rental_rate > (
5     select avg(f2.rental_rate) from film as f2
6     inner join film_category as fc2 on f2.film_id = fc2.film_id
7     where fc2.category_id = fc.category_id
8   );
```

The result grid shows the following data:

title	rental_rate
AMERICAN CIRCUS	4.99
ANTITRUST TOMATOES	2.99
BAREFOOT MANCHURIAN	2.99
BERETS AGENT	2.99
CAMPUS REMEMBER	2.99
CASUALTIES ENCINO	4.99
CLUELESS BUCKET	2.99
DARKO DORADO	4.99

Part B

I used NOT EXISTS to find customers who have never made a payment. The result shows no matching customer in this Sakila data snapshot.



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
1 • use sakila;
2 • select c.customer_id
3   from customer as c
4   where not exists (
5     select 1
6     from payment as p
7     where p.customer_id = c.customer_id
8   );
```

The result grid shows the following data:

customer_id
NULL

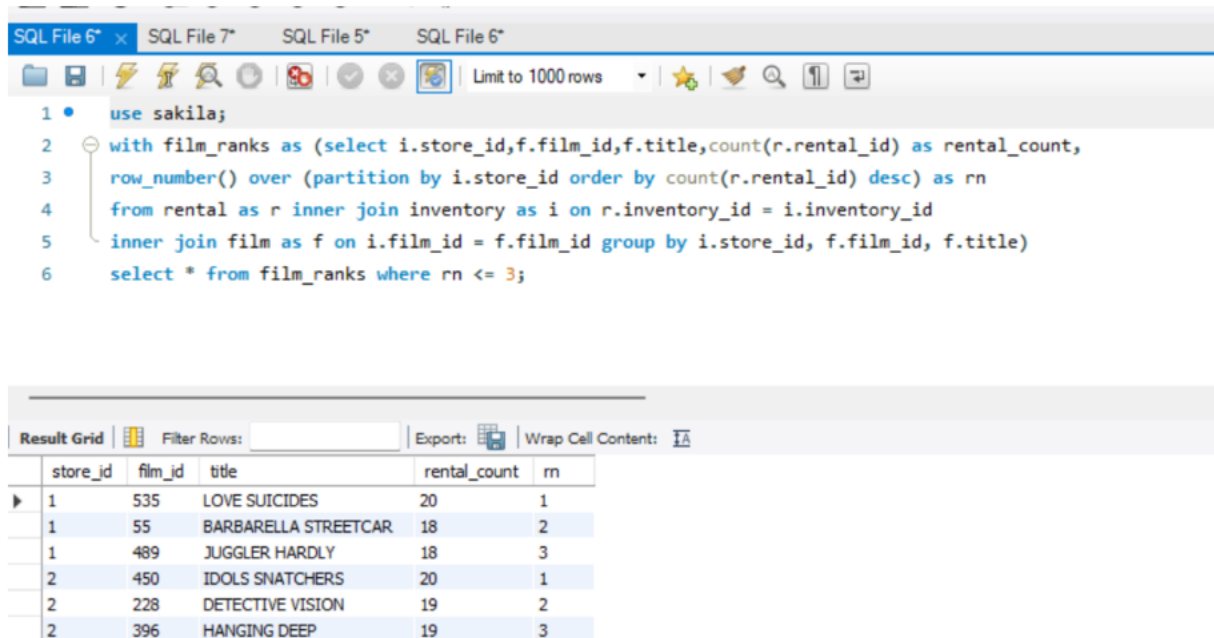
Part C

EXISTS can be more efficient for large subqueries because it can stop searching after finding a match, while IN may need to process more of the subquery result before completing the comparison.

Q20. Window Functions Store Level Analysis

Part A

I used a CTE to count rentals for each film at each store and then applied ROW_NUMBER partitioned by store_id and ordered by rental count in descending order. Filtering rn less than or equal to 3 returns the top three films for each store. The screenshot below shows the corrected query and result for the assignment.



The screenshot shows a SQL IDE with a query editor and a results grid. The query is as follows:

```
1 use sakila;
2 with film_ranks as (select i.store_id,f.film_id,f.title,count(r.rental_id) as rental_count,
3   row_number() over (partition by i.store_id order by count(r.rental_id) desc) as rn
4   from rental as r inner join inventory as i on r.inventory_id = i.inventory_id
5   inner join film as f on i.film_id = f.film_id group by i.store_id, f.film_id, f.title)
6 select * from film_ranks where rn <= 3;
```

The results grid shows the following data:

	store_id	film_id	title	rental_count	rn
▶	1	535	LOVE SUICIDES	20	1
	1	55	BARBARELLA STREETCAR	18	2
	1	489	JUGGLER HARDLY	18	3
	2	450	IDOLS SNATCHERS	20	1
	2	228	DETECTIVE VISION	19	2
	2	396	HANGING DEEP	19	3

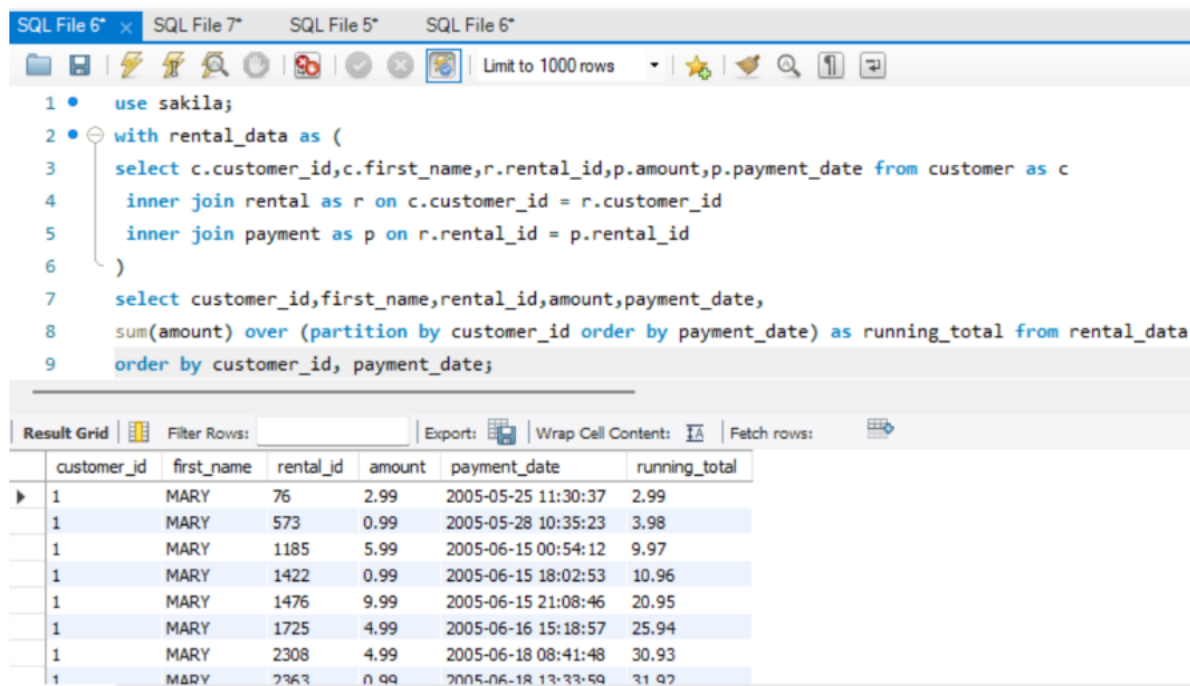
Part B

ROW_NUMBER() gives unique numbers; RANK() shares ties but leaves gaps; DENSE_RANK() shares ties without gaps.

Bonus. CTE, Join and Running Total

Part A

The bonus query combines a CTE with joins and SUM OVER to calculate each customer's running spending total in payment date order.



The screenshot shows a SQL IDE with a query editor and a result grid. The query uses a CTE named 'rental_data' to join the 'customer', 'rental', and 'payment' tables. It then uses a window function 'SUM OVER' to calculate a running total of the amount for each customer, ordered by payment date.

```
1 • use sakila;
2 • with rental_data as (
3   select c.customer_id,c.first_name,r.rental_id,p.amount,p.payment_date from customer as c
4   inner join rental as r on c.customer_id = r.customer_id
5   inner join payment as p on r.rental_id = p.rental_id
6 )
7 select customer_id,first_name,rental_id,amount,payment_date,
8 sum(amount) over (partition by customer_id order by payment_date) as running_total from rental_data
9 order by customer_id, payment_date;
```

The result grid displays the following data:

	customer_id	first_name	rental_id	amount	payment_date	running_total
▶	1	MARY	76	2.99	2005-05-25 11:30:37	2.99
	1	MARY	573	0.99	2005-05-28 10:35:23	3.98
	1	MARY	1185	5.99	2005-06-15 00:54:12	9.97
	1	MARY	1422	0.99	2005-06-15 18:02:53	10.96
	1	MARY	1476	9.99	2005-06-15 21:08:46	20.95
	1	MARY	1725	4.99	2005-06-16 15:18:57	25.94
	1	MARY	2308	4.99	2005-06-18 08:41:48	30.93
	1	MARY	2363	0.99	2005-06-18 13:33:59	31.92